Team Sentio

Raspberry Pi Setup and Run Guide

WRO Future Engineers 2026

Mechanical platform: V3
Open software: V5

Team Sentio | Robofun Lab (RFL), India

Purpose. This guide explains how to prepare a Raspberry Pi 5, verify the robot hardware, and run the Team Sentio Open Challenge program. It also records the current public-code status of the Obstacle and Parking programs so that the engineering journal does not claim more reproducibility than the repository currently provides.

Prepared for inclusion in the Team Sentio engineering documentation

Contents

1 Scope and safety

The competition vehicle uses a Raspberry Pi 5 (4 GB) running Raspberry Pi OS. The current Open Challenge program uses Picamera2, OpenCV, NumPy, and GPIO/PWM control. This document is intended for trained team members working with a real autonomous vehicle, a motor driver, a servo, and a 3S LiPo battery.

Hardware warning. The programs control real motors and steering. Keep the drivetrain lifted clear of the floor during the first hardware test, keep the emergency power switch accessible, and never connect the motor directly to a Raspberry Pi GPIO pin.

Do not continue testing if there is repeated Raspberry Pi rebooting, visible smoke, abnormal motor-driver heating, drivetrain binding, battery swelling, or uncertain battery polarity. LiPo charging must be supervised, with the correct cell-count setting and verified polarity.

2 Hardware connections

Connect the electronics according to the schematic in `schemes/`. Confirm a common ground between the Raspberry Pi, motor driver, servo supply, and sensor electronics before applying power. Check XT60 polarity before connecting the 3S LiPo. Connect the front Camera Module 3 used for navigation, the rear Camera Module 3 used for parking, the MPU6050, and the SSD1306 display.

BCM GPIO	Function	Connection / role
GPIO2	I2C SDA	MPU6050 and SSD1306 data line
GPIO3	I2C SCL	MPU6050 and SSD1306 clock line
GPIO5	Motor direction 1	TB6612FNG direction input 1
GPIO6	Motor direction 2	TB6612FNG direction input 2
GPIO13	Motor PWM	TB6612FNG motor-speed PWM, 1 kHz
GPIO22	Servo signal	DS3225 Ackermann steering servo, 50 Hz
GPIO18	Push-button input	Auxiliary schematic connection; verify use in final software
GPIO23 / GPIO12	Status LEDs	Auxiliary schematic connections; verify use in final software

The final challenge programs primarily use GPIO2/3, GPIO5/6/13, and GPIO22. Auxiliary GPIO assignments should remain in the guide only if the corresponding components are physically connected and used by the final software.

3 Install Raspberry Pi OS and dependencies

Use Raspberry Pi OS 64-bit on the Raspberry Pi 5. Raspberry Pi OS with Desktop is recommended for the present programs because they display diagnostic frames through `cv2.imshow()`.
After the first boot, open a terminal and update the system:

```
sudo apt update
sudo apt full-upgrade -y
```

Install the core software packages:

```
sudo apt install -y git python3-venv python3-opencv python3-numpy \
  python3-picamera2 i2c-tools
```

The code imports `RPi.GPIO`. Verify whether the GPIO interface is available:

```
python3 -c "import RPi.GPIO as GPIO; print('GPIO OK')"
```

On Raspberry Pi 5 systems where the original `RPi.GPIO` implementation is unavailable or incompatible, install the compatible implementation. Use one implementation or the other, not both in the same Python environment:

```
sudo apt install -y python3-rpi-lgpio
python3 -c "import RPi.GPIO as GPIO; print('GPIO OK')"
```

4 Enable I2C and verify the bus

Run the Raspberry Pi configuration utility:

```
sudo raspi-config
```

Open **Interface Options → I2C → Enable**, then reboot:

```
sudo reboot
```

After reboot, verify that the I2C bus responds:

```
i2cdetect -y 1
```

The connected MPU6050 and SSD1306 should appear on the bus before attempting any IMU-assisted parking or obstacle procedure.

5 Download the repository

Clone the repository and enter its root directory:

```
git clone https://github.com/teamsentiorobotics-svg/World-Robot-Olympiad---Team-Sentio-.git
cd World-Robot-Olympiad---Team-Sentio-
```

The current challenge programs are stored at:

```
src/Open_Challenge.py
src/Obstacle_Challenge.py
```

If the team publishes a dependency file, install it from the repository root with:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The dependency file and setup commands must be tested on a clean Raspberry Pi before being described as the official final installation method.

6 Verify the cameras

List cameras detected by Raspberry Pi OS:

```
rpicam-hello --list-cameras
```

Both Camera Module 3 units should be detected before testing parking. The current public programs instantiate `Picamera2()` without explicitly specifying a camera index. Therefore, physically identify the front navigation camera and confirm the captured view before enabling the drivetrain; do not assume enumeration order remains unchanged after reconnecting CSI cables.
The final physical camera positions are approximately:

Camera	Pose	Role
Front Camera 3	About 50° downward pitch; about 5 mm right of centre	Wall, marker, and obstacle perception
Rear Camera 3	About 45° downward pitch; approximately 0° yaw	Rear-wall and parking geometry

Camera geometry is part of calibration. Moving a camera can require retuning colour thresholds and wall-detection behavior.

7 Verify the software environment

Before applying power to the drivetrain, verify the core imports:

```
python3 -c "import cv2, numpy, RPi.GPIO; from picamera2 import Picamera2;
print('Core imports OK')"
```

A successful result is:

```
Core imports OK
```

The Open Challenge source currently imports OpenCV, NumPy, RPi.GPIO, and Picamera2. The Obstacle Challenge additionally requires its IMU helper module once that file is published in `src/`.

8 Pre-run checklist

Before every autonomous run, complete the following checks:

1. Check LiPo condition and polarity.

2. Confirm the XT60 connector is fully seated.
3. Confirm the drivetrain rotates freely.
4. Check that the steering linkage is attached correctly.
5. Confirm the wheels are approximately straight at the centre command.
6. Verify the intended camera is detected and showing the correct view.
7. Confirm the front camera angle has not moved.
8. Check motor direction with the driven wheels lifted clear.
9. Check marker and wall detection under current lighting.
10. Keep the main power switch accessible as an emergency cut-off.

9 Run the Open Challenge

The current Open Challenge command is:

```
python3 src/Open_Challenge.py
```

After startup, the program initializes GPIO and the camera, allows automatic exposure and white balance to settle for approximately two seconds, locks exposure and analogue gain, centres the steering, and starts the drivetrain automatically. The current Open V5 source commands `forward(100)`, equivalent to 100% PWM, so position the robot safely before the autonomous stage begins.

For the first hardware test, lift the driven wheels clear of the surface. For a competition run, place the robot in the correct starting position before executing the program.

The program prints messages such as:

```
Auto adjusting camera...
Camera locked
Robot Started
CLOCKWISE
```

The first valid coloured marker determines direction: blue first means anticlockwise, and orange first means clockwise. The first marker establishes direction but is not counted as a completed course event. The program then counts 12 valid crossings for the three-lap run.

To stop manually, click the OpenCV display window so it has keyboard focus, then press `q`. If the display is unavailable, use the physical emergency power cut rather than relying on keyboard input. The normal stop sequence centres the steering and stops the motor.

10 Run the Obstacle Challenge

The intended command is:

```
python3 src/Obstacle_Challenge.py
```

Public-code status. The current public GitHub snapshot must not yet be treated as the final reproducible Obstacle/Parking executable. This limitation is intentionally recorded here so the setup guide remains evidence-accurate.

At the current public snapshot:

- `Obstacle_Challenge.py` imports `from heading import MPU6050Heading`, but `heading.py` is not currently present in `src/`.
- Parking speed variables are currently `ps = 0` and `rs = 0`.
- The normal steering actuation line is commented in the displayed navigation loop.

Before final competition submission, the repository must include:

```
src/heading.py
```

and the exact tested Obstacle and Parking source used on the physical robot. Once those files are synchronized and verified, launch the program from the repository root with the command above. The final executable should reproduce the tested sequence: course navigation, red/green pillar avoidance, lap completion, parking detection, rear-camera/IMU-assisted positioning, and stop.

11 Camera startup and safe shutdown

Both current challenge programs allow approximately two seconds for automatic exposure and white-balance adjustment. The program records `ExposureTime` and `AnalogueGain`, then disables automatic exposure and white balance for the remainder of the run. For reproducible results, avoid moving the robot or dramatically changing the lighting during this startup period.

After testing, stop the program normally and shut down Raspberry Pi OS before disconnecting main power when practical:

```
sudo shutdown -h now
```

Wait for the Raspberry Pi to shut down before removing power. The physical main switch remains the immediate emergency cut-off if the robot behaves unexpectedly.

12 Final repository placement

The recommended repository structure is:

```
README.md                          quick-start summary
requirements.txt                   root-level dependencies
src/
  Open_Challenge.py
  Obstacle_Challenge.py
  heading.py                        required by Obstacle/Parking
other/
  PI_SETUP_AND_RUN.md               detailed version of this guide
```

```
Logic design.pdf
Parking Logic Designing.png
```

The root `README.md` should contain a short quick-start link to `other/PI_SETUP_AND_RUN.md`. Keep `requirements.txt` at the repository root so judges and standard tools can find it immediately.

13 Final verification checklist

Before describing the system as fully reproducible, a new team member should be able to clone the repository, install the dependencies, verify cameras and I2C, run the Open Challenge, run the Obstacle Challenge, and execute Parking without requesting missing files or undocumented commands.

Verification item	Completion condition
Repository files	`heading.py`, final obstacle/parking source, requirements/setup files are public
Hardware	Polarity, common ground, motor-driver path, cameras, IMU, and emergency cut-off verified
Software	Core imports pass and Open/Obstacle/Parking commands run from a clean setup
Calibration	Camera poses, steering limits, lighting, and I2C devices verified
Evidence	Timing sheet, video index, and final release identifier match the engineering journal

Document prepared for Team Sentio WRO Future Engineers 2026 engineering documentation.